

Module 11 — Core Skills Drill: Instrument a Toy FastAPI Service

Due: Tonight, after the concept lecture. Resubmissions are accepted through Saturday of the assignment week.

You learned in the reading and the concept lecture that `/metrics` is a contract — counters, histograms, and gauges describe a service's *fast* and *reliable* axes the same way Pydantic models describe its request/response shape. Tonight's drill moves you from *recognizing* the `prometheus_client` API to *applying* it: take a two-endpoint toy service, add three middlewares (request-id, structured logging, metrics), declare three metrics (counter, histogram, gauge), mount `/metrics`, and write a self-test that GETs `/metrics` and confirms the families are present and the counter increments after traffic.

The drill is deliberately not the Module 10 stack. The toy is two endpoints, in-memory, no Docker. The instrumentation pattern is identical to what Tuesday's Lab will require — practice on a clean surface so you build the *pattern* rather than copy your eventual Lab code from your drill.

Setup

Accept the drill template repo at `https://github.com/LevelUp-Applied-AI/m11-d11` — click “Use this template” → “Create a new repository”. On the create-new-repository page, name your copy `m11-d11`, leave Visibility as Public, and create the repository under your own GitHub account. Then clone your new fork:

```
git clone https://github.com/<your-username>/m11-d11.git
cd m11-d11
git checkout -b drill-11-instrument-toy-service
```

Use **Python 3.11** for this template (the pinned `pydantic==2.6.0` does not build on 3.13). If your default Python is newer, create a 3.11 venv: `python3.11 -m venv .venv && source .venv/bin/activate`.

Install dependencies (cached from the M11 reading — should be fast):

```
pip install -r requirements.txt
```

If you have not yet installed `prometheus-client`, go back to the [Setup: install before the concept session](#) section of the reading and complete the install before proceeding.

The starter ships:

- `app.py` — a FastAPI app with two endpoints (`POST /echo`, `GET /sum`) and TODO bodies for the middlewares and metrics.
- `tests/test_drill_autograde.py` — the autograder (do not modify).
- `tests/test_drill.py` — your self-test, with `pytest.fail("Not implemented")` placeholders.
- `requirements.txt` — `fastapi`, `uvicorn`, `pydantic`, `httpx`, `prometheus-client`, `pytest`.

What to do

Task 1 — Declare three metrics in `app.py`

At module scope, declare:

- `requests_total` — a `Counter` named `requests_total` with help `"Total HTTP requests"` and labels `["path", "status"]`.
- `request_latency_seconds` — a `Histogram` named `request_latency_seconds` with help `"Request latency in seconds"`, labels `["path"]`, and the explicit bucket sequence `[0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10]`. (`prometheus_client`'s own `DEFAULT_BUCKETS` adds a few in-between boundaries — `[0.075, 0.75, 7.5]` — and a `+Inf` sentinel; the simpler sequence above is what the Lab and Drill use so output is easier to read on a small toy service.)
- `inflight_requests` — a `Gauge` named `inflight_requests` with help `"In-flight requests"` and no labels.

Task 2 — Wire three middlewares

The starter has TODO markers for three middlewares — you write the classes yourself from scratch. Implement each, then add them to the app in the correct order. Order matters: middlewares are added *innermost-last* in Starlette, so call `app.add_middleware(MetricsMiddleware)` first, then `StructuredLoggingMiddleware`, then `RequestIdMiddleware`. The request hits the request-id middleware first (outermost) and the metrics middleware last (innermost, closest to the route).

Request-id middleware. Generate `uuid.uuid4().hex`, store it in a module-level `contextvars.ContextVar` named `request_id_var` (declare it with default `""` so a read before any request returns a safe empty string), set the `X-Request-ID` response header to the same id.

Structured-logging middleware. Time the request. Emit one JSON log line at INFO level **through the logging module** (e.g. `logging.getLogger("app").info(json.dumps({...}))`); the autograder reads `caplog`, which only captures records emitted via `logging`. The autograder requires four keys in the line — `request_id` (read from `request_id_var.get()`), `path`, `status`, `latency_ms`. Add `ts` (timestamp) and `level` (`"INFO"`) too as best practice. Use `json.dumps(...)` to format — never `print(...)` and never write to `sys.stderr` directly (caplog will not see those).

Metrics middleware. Increment `inflight_requests` on entry; decrement on exit (use `try/finally`). Time the request. After the response, increment `requests_total.labels(path=request.url.path, status=str(response.status_code)).inc()` and call `request_latency_seconds.labels(path=request.url.path).observe(elapsed_seconds)`.

Task 3 — Mount `/metrics`

After the middleware wiring, mount the metrics ASGI (Asynchronous Server Gateway Interface) app:

```
from prometheus_client import make_asgi_app
app.mount("/metrics", make_asgi_app())
```

Task 4 — Write your self-test in `tests/test_drill.py`

Replace every `pytest.fail("Not implemented")` placeholder with a real test. Use FastAPI's `TestClient`. At least **four** test functions, each with at least one `assert`. Required coverage:

- After 3 calls to `POST /echo` and 2 calls to `GET /sum`, GETting `/metrics` returns 200.
- The `/metrics` body contains the three metric names: `requests_total`, `request_latency_seconds`, `inflight_requests`.
- The `/metrics` body contains a line for `requests_total{path="/echo",status="200"}` with value ≥ 3 .
- Every non-`/metrics` response has an `X-Request-ID` response header that is non-empty.

Run your tests:

```
pytest tests/test_drill.py -v
```

All your tests must pass before you push.

Task 5 — Open a PR

Commit, push, and open a PR from your branch into your fork's `main`:

```
git add -A
git commit -m "Drill 11 - instrument toy service"
git push -u origin drill-11-instrument-toy-service
```

In GitHub, open a PR within your fork. Your PR description must include:

- Confirmation that `pytest tests/ -v` passes locally (paste the summary line).
- The first 20 lines of your `/metrics` output (run the toy with `uvicorn app:app --port 8001`, make a few calls, then `curl -L localhost:8001/metrics | head -20`). The `-L` is required because `app.mount("/metrics", make_asgi_app())` serves the metrics at `/metrics/` (trailing slash) and Starlette responds to `/metrics` with a 307 redirect; without `-L` you get an empty body and a misleading `requests_total{path="/metrics",status="307"}` line.
- Paste your PR URL into TalentLMS → Module 11 → Drill 11 to submit this assignment.

Common pitfalls

- Middleware order reversed.** If your metrics middleware runs outermost, the latency you record includes time spent in the other middlewares — small, but the design intent is to time the *handler*.
- `.inc()` **called on the parent metric, not a labelled child.** `requests_total.inc()` raises when labels are declared. Always `requests_total.labels(path=..., status=...).inc()`.
- Gauge missing `try/finally`.** If the handler raises, `inflight_requests.dec()` never runs and the gauge drifts up forever.
- Structured log uses `print(...)`.** The autograder reads the captured log stream — `print` goes to stdout, not to the `logging` framework. Use `logging.getLogger("app").info(json.dumps({...}))`.
- `X-Request-ID` set on the request, not the response.** The header goes on `response.headers`, not on `request.headers`.